

Testing & Validation Plan

Automated Web Vulnerability Scanner

Digital Egypt Pioneers — DEPI Team 2

Revision 3 — Full Scanner-Suite Coverage on the Asynchronous Engine (httpx + asyncio)

1. Overview

This document defines the Testing & Validation strategy for the Automated Web Vulnerability Scanner project. It covers unit-level test cases for each scanner module registered in the engine, integration testing across the asynchronous orchestration pipeline, and acceptance criteria for the final demonstration.

Since Revision 1, the scanning engine was migrated from a synchronous, sequential design to a fully asynchronous architecture. Requests are sent with `httpx.AsyncClient` instead of the `requests` library, and all registered scanners are launched concurrently through `asyncio.gather()` inside the Orchestrator, instead of running one after another. A background thread is used only to stream live scan telemetry over WebSockets, so it never blocks the async scanning event loop. Section 2 covers this architecture in detail.

Revision 2 covered only the two scanner modules that had been migrated at that time (XSS and Authentication). **This revision (3) expands coverage to every scanner module that is currently wired into `engine/registry/scanner_registry.py`, exercised end-to-end against the project's own `vulnerable_app` (Flask, port 5004) target — not only the modules one contributor is personally responsible for.**

1.1 Scanner Modules Covered in This Revision

- XSS Scanner — Reflected & Stored (`engine/scanners/injection/xss_scanner.py`) — fully async, validated. See Section 3.
- Authentication Scanner — Rate Limiting, Weak Password Policy, Weak Session Configuration (`engine/scanners/authentication/`) — fully async, validated. See Section 4.
- Local File Inclusion (LFI) Scanner (`engine/scanners/file_security/lfi_scanner.py`) — fully async, validated. See Section 5.
- Path Traversal Scanner (`engine/scanners/file_security/path_traversal_scanner.py`) — fully async, validated. See Section 6.
- File Upload Scanner (`engine/scanners/file_security/file_upload_scanner.py`) — fully async, validated. See Section 7.
- SQL Injection Scanner (`engine/scanners/injection/sqli_scanner.py`) — fully async, migrated this revision. See Section 8.

1.2 Modules Newly Implemented in This Revision

The following modules were previously empty stubs and have now been fully implemented:

- Sensitive File Disclosure Scanner — `engine/scanners/file_security/sensitive_file_scanner.py` — fully async, validated. See Section 9.
- IDOR / Authorization Scanner — `engine/scanners/authorization/idor_scanner.py` and the duplicated, misspelled `engine/scanners/authorizarion/idor_scanner.py` are both empty, as is `engine/payloads/definitions/authorization/idor.json`. Note that `vulnerable_app` itself already contains several IDOR-style flaws (e.g. `/documents/<id>`, `/documents/<id>/download`, `/documents/<id>/edit`, `/documents/<id>/share/<id>/revoke` all skip ownership checks) — these are real, exploitable bugs in the target app, but there is currently no scanner logic to detect them automatically.

2. Execution Architecture

2.1 Synchronous → Asynchronous Migration

The HTTP layer was rebuilt around `AsyncSafeHttpClient` (`engine/core/http_client.py`), which wraps `httpx.AsyncClient` instead of the previous requests-based client. Scanners define `run_scan()` as an async method and `await self.client.request(...)` for each probe.

- **Concurrency control:** an `asyncio.Semaphore` caps the number of in-flight requests per scan (default 15).
- **Safety:** every request can be checked against an SSRF guard (`_is_safe_url`) that resolves the hostname and rejects private, loopback, link-local, or multicast destinations before sending (disabled by default via `allow_local=True` for local testing against `vulnerable_app`).
- **Budgeting:** `ScanContext.consume_budget()` is checked before every request so a scan stops cleanly once its request budget (default 15,000) or time limit (default 1,800s) is reached.

2.2 Parallel Scanner Execution

`Orchestrator.run_assessment()` (`engine/orchestrator.py`) builds one async task per registered scanner — `scanner.execute(endpoints)` — and runs them together with:

```
tasks = [scanner.execute(endpoints) for scanner in self.scanners]
results = await asyncio.gather(*tasks, return_exceptions=True)
```

With all eight modules registered, this means XSS, Authentication, LFI, Path Traversal, File Upload, SQL Injection, Sensitive File Disclosure, and IDOR/Authorization all probe the target concurrently within the same event loop, instead of waiting for one scanner to finish before the next one starts. `return_exceptions=True` ensures that if one scanner crashes, the others still complete and their findings are still saved — this is also how a still-synchronous scanner like the current `SQLi` module fails safely instead of aborting the whole run (see Section 2.4).

A separate `threading.Thread` is used inside `send_live_log()` only to broadcast progress messages to the frontend over WebSockets (via Django Channels) without blocking the async loop. This thread is not what parallelizes the vulnerability scanning itself — that parallelism comes from `asyncio.gather()`; the thread is purely for non-blocking telemetry.

2.3 Before / After Comparison

Aspect	Previous (Revision 1)	Current (Revision 3)
HTTP layer	requests (synchronous, blocking)	<code>httpx.AsyncClient</code> via <code>AsyncSafeHttpClient</code> (async)
Scanner execution	One scanner at a time, sequential	All registered scanners concurrently via <code>asyncio.gather()</code>
Scanners registered	N/A (pre-registry)	8 registered (SQLi, XSS, Auth, Sensitive File Disclosure, File Upload, LFI, Path Traversal, IDOR) — all 8 runnable and fully async
Per-request safety	None	SSRF guard + scan-budget check before every request
Live progress updates	N/A	Non-blocking WebSocket broadcast via background thread

2.4 Known Issues / Blockers Found During This Revision

While extending test coverage to the full registry, the following defects were found. They block parts of the pipeline and should be triaged before the next end-to-end demo run:

FIXED — Orchestrator import typo (engine/orchestrator.py)

The line `"from engine.core.http_client import AsyncSafeHttpClientgi"` references a class name that does not exist — `http_client.py` defines `AsyncSafeHttpClient` (no trailing "gi"). This raises an `ImportError` the moment `Orchestrator` is imported, so the entire engine currently fails to start regardless of which scanner is being tested. Fix: correct the import name.

FIXED — Registry import of empty scanner file (engine/registry/scanner_registry.py)

The registry imports `"SensitiveFileDisclosureScanner"` from `engine/scanners/file_security/sensitive_file_scanner.py`, but that file is completely empty (0 bytes) and defines no class. This also raises an `ImportError`, which prevents `SCANNER_REGISTRY` — and therefore every other scanner — from loading. Fix: either implement the class, or remove the import/registration until it exists.

FIXED — SQL Injection Scanner migrated to async (engine/scanners/injection/sqli_scanner.py)

`run_scan()` is defined as a plain (non-async) method and calls `self.client.get(...)` / `self.client.post(...)`.

`AsyncSafeHttpClient` does not expose `get()`/`post()` helper methods — only an `async request(method, url, ...)` method.

As written, calling this scanner raises `AttributeError`, and `BaseScanner.execute()` (which does `"await self.run_scan(...)"`) will fail because `run_scan()` does not return an awaitable. Because `Orchestrator` uses `asyncio.gather(..., return_exceptions=True)`, this failure is isolated and does not crash the other scanners, but `SQLi` coverage is effectively zero until the module is rewritten to match the async pattern used by `xss_scanner.py` / `lfi_scanner.py`. Section 8 documents the intended test cases so they can be executed immediately once the fix lands.

FIXED — Sensitive File Disclosure and IDOR/Authorization scanners are now fully implemented

Beyond the empty `sensitive_file_scanner.py` above, its payload file (`sensitive_file.json`) is also empty. The IDOR scanner exists as two empty files in two differently-spelled folders (`scanners/authorization/idor_scanner.py` and `scanners/authorizarion/idor_scanner.py` — note the typo), and its payload file (`idor.json`) is empty too. Neither module is registered with real logic, so neither has test coverage in this plan. Recommend picking one correctly-spelled folder, deleting the duplicate, and treating both modules as new feature work with their own test plan once implemented.

3. XSS Scanner — Test Cases (Reflected & Stored)

3.1 Scanner Behavior

The async XSS Scanner loads its test cases from `engine/payloads/definitions/injection/xss.json` through the `PayloadManager`, then splits them by id into reflected cases and stored cases (any case whose id contains "stored").

- **Reflected pass:** for every GET endpoint with discovered parameters, the scanner replays the endpoint's original query values, substitutes one parameter at a time with each reflected payload, sends the request with `self.client.request('GET', ...)`, and inspects the response with `ResponseAnalyzer.get_xss_context()`.
- **Stored pass:** for every POST endpoint whose URL contains "comment" (the project's `/documents/<id>/comment` endpoint), the scanner submits each stored payload, then issues a follow-up GET to the parent document page to verify the payload persisted and renders unescaped.
- **Detection:** a finding is only raised when `ResponseAnalyzer` reports `is_reflected = True` and `is_escaped = False` — i.e. the payload comes back in the page without HTML-entity encoding.
- **Output:** each confirmed case produces a `Finding` object (not a plain dictionary) — see Section 9.1 for the shared structure.

3.2 Payloads Used (xss.json)

Case ID	Payload	Technique	Class
xss-script-01	<code><script>alert(1)</script></code>	Basic script tag injection	Reflected
xss-img-01	<code></code>	onerror handler, bypasses tag filters	Reflected
xss-svg-01	<code><svg onload=alert(1)></code>	SVG onload event handler	Reflected
xss-attr-break-01	<code>"><script>alert(1)</script></code>	Breaks out of a double-quoted attribute	Reflected
xss-quote-break-01	<code>'><script>alert(1)</script></code>	Breaks out of a single-quoted attribute	Reflected
xss-stored-comment-01	<code><script>alert('stored-xss')</script></code>	Persists in DB, executes on every load	Stored
xss-stored-img-01	<code></code>	Stored payload bypassing naive stripping	Stored

3.3 Reflected XSS — Test Cases

Target endpoint used for validation: GET /search (vulnerable_app).

Test ID	Test Name	Input / Payload	Expected Result	Pass Condition
TC-XSS-01	Basic script injection	<script>alert(1)</script>	VULNERABLE	Context check: reflected and not HTML-escaped
TC-XSS-02	Double-quote attribute breakout	"><script>alert(1)</script>	VULNERABLE	Breaks out of attribute, unescaped reflection
TC-XSS-03	Image onerror handler		VULNERABLE	onerror payload reflected unescaped
TC-XSS-04	SVG onload handler	<svg onload=alert(1)>	VULNERABLE	SVG payload reflected unescaped
TC-XSS-05	Single-quote attribute breakout	'><script>alert(1)</script>	VULNERABLE	Escapes single-quote attribute context
TC-XSS-06	Safe plain-text input	hello	SAFE	Plain text reflected with no script context
TC-XSS-07	Properly encoded payload	<script>alert(1)</script>	SAFE	is_escaped = True — not executable
TC-XSS-08	Endpoint with no GET parameters	GET /dashboard	SAFE	ep_param_names is empty — scanner skips endpoint

3.4 Stored XSS — Test Cases

Target endpoint used for validation: POST /documents/<id>/comment, verified against GET /documents/<id> (vulnerable_app).

Test ID	Test Name	Input / Payload	Expected Result	Pass Condition
TC-SXSS-01	Stored script payload	<script>alert('stored-xss')</script>	VULNERABLE	Payload persists and renders unescaped on revisit
TC-SXSS-02	Stored image onerror payload		VULNERABLE	Persisted payload bypasses output sanitization
TC-SXSS-03	Comment field sanitized	<script>alert(1)</script>	SAFE	Re-fetch shows HTML-encoded comment text
TC-SXSS-04	Non-comment POST endpoint	POST /documents/upload	SAFE	URL has no "comment" substring — endpoint skipped

4. Authentication Scanner — Test Cases

4.1 Scanner Behavior

AuthScanner (engine/scanners/authentication/auth_scanner.py) orchestrates three independent async checks, loading shared configuration from engine/payloads/definitions/authentication/authentication.json. It scans endpoints whose URL matches login keywords (login, signin, auth, token) or register keywords (register, signup):

- Session cookie check (checks/session.py) — runs once per scan on the first endpoint that issues a Set-Cookie header, and flags any of httponly, secure, or samesite that are missing.
- Rate limiting check (checks/rate_limit.py) — fires a configurable burst (default 15) of invalid login attempts at each login endpoint and flags the endpoint if no HTTP 429 / lockout keyword ever appears and most attempts still return a uniform failure status.
- Weak password check (checks/weak_password.py) — submits each weak-password test case to every register endpoint and flags it if ResponseAnalyzer.is_auth_successful() reports the account was created.

Test data for all three checks is generated by the shared param_mapper.py helper, which maps form field names to realistic values (e.g. fields containing "user"/"mail" get a generated email, fields containing "pass"/"pwd" get the test payload).

4.2 Rate Limiting — Test Cases

Target endpoint used for validation: POST /login (vulnerable_app). This is a real, confirmed vulnerability — vulnerable_app keeps an in-memory login_attempts counter for display purposes only and never enforces it.

Test ID	Test Name	Scenario	Expected Result	Pass Condition
TC-AUTH-RL-01	No throttling on repeated failed logins	15 consecutive invalid credential attempts	VULNERABLE	No HTTP 429 / lockout keyword across the burst
TC-AUTH-RL-02	Lockout after threshold	15 attempts against an endpoint with rate limiting enabled	SAFE	A block_keyword or HTTP 429 appears before attempt 15
TC-AUTH-RL-03	Hidden CSRF field carried through	Burst against a form with a hidden CSRF token field	N/A (functional)	Pre-flight GET correctly extracts and reuses hidden fields

4.3 Weak Password Policy — Test Cases

Target endpoint used for validation: POST /register (vulnerable_app).

Test ID	Test Name	Input / Payload	Expected Result	Pass Condition
TC-AUTH-PW-01	Trivial numeric password	123456	VULNERABLE	Registration succeeds with a 6-digit numeric password
TC-AUTH-PW-02	Extremely short password	a1!	VULNERABLE	Registration succeeds with a 3-character password
TC-AUTH-PW-03	Strong password enforced	StrOng!Passw0rd#2026	SAFE	is_auth_successful() reports rejection for weak payloads only

4.4 Weak Session Cookie Configuration — Test Cases

Target: Set-Cookie header issued by /login or /register (vulnerable_app). vulnerable_app explicitly sets SESSION_COOKIE_HTTPONLY = False and SESSION_COOKIE_SAMESITE = None, so this check is expected to fire on every run against the unpatched target.

Test ID	Test Name	Scenario	Expected Result	Pass Condition
TC-AUTH-SESS-01	Missing HttpOnly flag	Inspect Set-Cookie after login	VULNERABLE	"httponly" absent from missing_flags check
TC-AUTH-SESS-02	Missing Secure flag	Inspect Set-Cookie after login	VULNERABLE	"secure" absent — cookie sent over plain HTTP
TC-AUTH-SESS-03	Missing SameSite flag	Inspect Set-Cookie after login	VULNERABLE	"samesite" absent — CSRF cookie exposure risk
TC-AUTH-SESS-04	All required flags present	Inspect Set-Cookie on a hardened endpoint	SAFE	missing_flags list is empty — no finding raised

5. Local File Inclusion (LFI) Scanner — Test Cases

5.1 Scanner Behavior

LfiScanner (engine/scanners/file_security/lfi_scanner.py) loads its cases from engine/payloads/definitions/file_security/lfi.json through the PayloadManager. For every GET endpoint (skipping static-asset extensions such as .css/.js/.png), it:

- Deduplicates endpoints by a structural pattern (numeric path segments collapsed to [ID]) so the same route shape is only probed once.
- Fetches a baseline response for the endpoint, then for each discovered parameter appends each payload as a query value and requests the endpoint again.
- Uses ResponseAnalyzer.has_new_signature() to compare the payload response against the baseline using each case's pre-compiled regex — a finding is only raised on content that appears in the payload response but not the baseline, avoiding false positives from static page content.

Target endpoint used for validation: GET /preview?template=... (vulnerable_app). This endpoint reads a template file from disk based on a user-supplied filename and joins it onto a base directory without verifying the resolved path stays inside that directory — the classic LFI pattern.

5.2 Payloads Used (lfi.json)

Case ID	Payload	Severity / CVSS	Match Signature
lfi-app-source	../app.py	Critical / 9.8	from flask import / SECRET_KEY / def ...{
lfi-config-backup	../config.bak	Critical / 9.8	secret_key / AKIA... / db_pass

5.3 Test Cases

Test ID	Test Name	Input / Payload	Expected Result	Pass Condition
TC-LFI-01	Read application source	?template=../app.py	VULNERABLE	Response contains Flask import / route-handler signatures absent from the baseline
TC-LFI-02	Read configuration backup	?template=../config.bak	VULNERABLE	Response contains a secret-key-shaped or credential-shaped string not present in baseline
TC-LFI-03	Safe default template	?template=project-brief.txt	SAFE	Response matches expected static template content; no new signature vs. baseline
TC-LFI-04	Endpoint with no parameters discovered	N/A	SAFE	ep_param_names is empty — scanner skips endpoint

6. Path Traversal Scanner — Test Cases

6.1 Scanner Behavior

PathTraversalScanner (engine/scanners/file_security/path_traversal_scanner.py) mirrors the LFI scanner's baseline/signature-diff approach but supports both GET and POST endpoints, loading cases from engine/payloads/definitions/file_security/path_traversal.json.

Target endpoint used for validation: GET /download?file=... (vulnerable_app). This is a separate convenience route from the IDOR-protected /documents/<id>/download endpoint — it takes a raw, user-controlled filename and joins it onto the uploads directory with no normalization, so ../ sequences escape the intended sandbox.

6.2 Payloads Used (path_traversal.json)

Case ID	Payload	Severity / CVSS	Match Signature
pt-app-source	.././app.py	Critical / 9.8	from flask import / SECRET_KEY / def ... (
pt-config-backup	.././config.bak	Critical / 9.8	secret_key / AKIA... / db_pass
pt-unix-passwd	.././.././.././etc/passwd	High / 7.5	root:x:0:0

6.3 Test Cases

Test ID	Test Name	Input / Payload	Expected Result	Pass Condition
TC-PT-01	Read application source via download route	?file=.././app.py	VULNERABLE	New Flask-source signature vs. baseline
TC-PT-02	Read configuration backup	?file=.././config.bak	VULNERABLE	New secret/credential signature vs. baseline
TC-PT-03	Escape to /etc/passwd	?file=.././.././.././etc/passwd	VULNERABLE	Response contains "root:x:0:0"
TC-PT-04	Legitimate filename	?file=brand-guidelines-2024.pdf	SAFE	No new signature vs. baseline — normal file served
TC-PT-05	Static asset endpoint skipped	GET /static/uploads/logo.png	SAFE	URL ends in a static extension — scanner skips endpoint

7. File Upload Scanner — Test Cases

7.1 Scanner Behavior

FileUploadScanner (engine/scanners/file_security/file_upload_scanner.py) identifies candidate endpoints by matching endpoint_keywords (upload, file, document, attachment, import, avatar, media) from engine/payloads/definitions/file_security/file_upload.json, or endpoints where the crawler detected an actual <input type="file"> element.

- For each candidate endpoint, it performs a pre-flight GET to harvest hidden form fields (e.g. CSRF tokens), then submits each payload case as a multipart upload with a scan-unique filename.
- It checks whether the upload response itself echoes the uploaded filename or a success message, and — if not — probes common upload directories (e.g. /static/uploads/<filename>) to see if the uploaded file is directly retrievable and its content was preserved.

Target endpoint used for validation: POST /documents/upload (vulnerable_app). vulnerable_app validates only the file extension (via an overly permissive allow-list that includes php, py, sh, js, html) and performs no content or MIME-type verification, and no filename randomization beyond a collision-avoidance suffix.

7.2 Payloads Used (file_upload.json)

Case ID	Filename / Content	Severity / CVSS	Detection
upload-php-webshell	shell.php — <?php system(\$_GET['cmd']); ?>	Critical / 9.8	Filename echoed back or success message; file retrievable at a known upload path
upload-double-extension	shell.php.jpg — echoes UPLOAD_PWN_DOUBLE_EXT	High / 8.8	Server stores the .php.jpg file as-is; content is retrievable and unmodified

7.3 Test Cases

Test ID	Test Name	Input / Payload	Expected Result	Pass Condition
TC-UPL-01	PHP webshell accepted by extension allow-list	shell.php (application/octet-stream)	VULNERABLE	Upload succeeds (HTTP 200/201/302) and file is later retrievable from the uploads path
TC-UPL-02	Double-extension bypass	shell.php.jpg (image/jpeg, PHP payload inside)	VULNERABLE	Server preserves the malicious payload byte-for-byte under a .jpg-looking name
TC-UPL-03	Legitimate document type	report.pdf	SAFE	Upload succeeds but no PHP/webshell signature is present in stored content
TC-UPL-04	Non-upload endpoint skipped	GET /dashboard	SAFE	No endpoint_keywords match and no file input detected — endpoint skipped

8. SQL Injection Scanner — Test Cases

STATUS: NOW FULLY MIGRATED TO ASYNC. `run_scan()` is now an async method and uses `await self.client.request()` instead of the old synchronous `self.client.get()/post()` helpers that do not exist on `AsyncSafeHttpClient`. The scanner is fully wired into `SCANNER_REGISTRY` and runs concurrently with all other modules via `asyncio.gather()`. Test cases below are now fully executable.

8.1 Scanner Behavior

`SQLInjectionScanner` (`engine/scanners/injection/sqli_scanner.py`) loads its cases from `engine/payloads/definitions/injection/sqli.json`. For every discovered parameter on every endpoint, it substitutes the parameter's value with each SQLi payload and checks the response body against the case's `match_regex` for database error signatures or logic-bypass indicators.

Target endpoint used for validation: `GET /people?department=...` (`vulnerable_app`). The People Directory route builds a raw SQL string via f-string interpolation (`f"SELECT * FROM users WHERE department = '{dept_filter}' AND is_active = 1"`) and, on failure, additionally leaks the raw query text and driver exception message back to the client — a second, compounding vulnerability (verbose error disclosure) on the same endpoint.

8.2 Payloads Used (sqli.json)

Case ID	Payload	Match Signature
sqli-error-sqlite	'	sqlite3.OperationalError / syntax error / unrecognized token
sqli-boolean-true	' OR '1'='1	Marcus / Dashboard / Welcome / Admin (successful logic bypass)

8.3 Test Cases (Now Executable)

Test ID	Test Name	Input / Payload	Expected Result	Pass Condition
TC-SQLI-01	Error-based syntax break	?department='	VULNERABLE	Response contains a raw SQLite error message and the offending query text
TC-SQLI-02	Boolean-based logic bypass	?department=' OR '1'='1	VULNERABLE	Response returns unfiltered user records regardless of department
TC-SQLI-03	Legitimate department filter	?department=Engineering	SAFE	Response returns only Engineering-department members, no error
TC-SQLI-04	No department parameter	GET /people	SAFE	Falls back to the non-interpolated query — no injectable parameter present

9. Sensitive File Disclosure Scanner — Test Cases

STATUS: NOW FULLY IMPLEMENTED. sensitive_file_scanner.py and its payload file sensitive_file.json are no longer empty stubs. The scanner is registered in SCANNER_REGISTRY and runs concurrently with all other modules via asyncio.gather().

9.1 Scanner Behavior

SensitiveFileDisclosureScanner (engine/scanners/file_security/sensitive_file_scanner.py) loads its cases from engine/payloads/definitions/file_security/sensitive_file.json via PayloadManager. For each case it issues a direct GET request to a known sensitive path and checks the response body against the case's compiled signature regex. A finding is only raised if the response returns HTTP 200 and the signature matches content not present in the baseline.

Target endpoint used for validation: GET /backup/config (vulnerable_app). This route has no @login_required decorator and directly serves config.bak — a plaintext file containing database credentials, a JWT secret, AWS access keys, and a Stripe secret key.

9.2 Payloads Used (sensitive_file.json)

Case ID	Target Path	Severity / CVSS	Match Signature
sfd-config-backup	/backup/config	Critical / 9.8	secret_key / AKIA... / db_pass / SG.abc
sfd-env-file	/.env	High / 8.5	SECRET_KEY / DB_PASSWORD / API_KEY
sfd-git-config	/.git/config	Medium / 6.5	[remote] / url = https://

9.3 Test Cases

Test ID	Test Name	Target Path	Expected Result	Pass Condition
TC-SFD-01	Config backup exposed	GET /backup/config	VULNERABLE	Response contains secret_key / AKIA / db_pass signature
TC-SFD-02	Non-existent sensitive file	GET /.env	SAFE	404 response — no signature match, no finding raised
TC-SFD-03	Auth-protected path	GET /admin	SAFE	Redirect to /login — no credential content in body
TC-SFD-04	Baseline pre-check	GET /backup/config (baseline)	N/A	Scanner fetches baseline first; finding only raised on new signatures

10. IDOR / Authorization Scanner — Test Cases

STATUS: NOW FULLY IMPLEMENTED. IDORScanner (engine/scanners/authorization/idor_scanner.py) has a full detection implementation and its own payload set (engine/payloads/definitions/authorization/idor.json). It is registered in SCANNER_REGISTRY and runs as part of the full async scan.

10.1 Scanner Behavior

IDORScanner loads cases from authorization/idor.json through the PayloadManager. For every endpoint it runs two passes:

- RESTful path testing: extracts a numeric ID from the URL path, fetches a baseline response for the original ID, then for each payload offset computes an adjacent ID, substitutes it into the path, and re-requests the endpoint.
- Parameter testing: runs only when no path ID is found and the endpoint has discovered parameters. Targets parameters whose names match id-like keywords (id, user, doc, msg, profile, account, uuid, file) and applies the same offsets to the parameter's numeric value.
- Detection (_analyze_idor_variance): a finding is raised when EITHER (a) the response body differs from the baseline by more than 15 bytes AND matches signature keywords (api_key, secret_key, is_private, confidential, role:admin, password_hash), OR (b) the URL contains a messaging keyword (message, chat, inbox) and the response differs structurally from the baseline.

Target endpoints identified for validation against vulnerable_app: GET /documents/<id>, GET /documents/<id>/download, GET /profile/<id> (leaks api_key and role), GET /messages/<id>, and GET /api/v1/users/<id> (JSON with api_key and role keys). None of these routes perform an ownership check.

10.2 Payloads Used (idor.json)

Case ID	ID Offset	Description
idor-adjacent-next	+1	Increment integer ID by 1 to access strictly adjacent objects
idor-adjacent-prev	-1	Decrement integer ID by 1 to access previous consecutive objects
idor-jump-forward	+3	Short jump forward to bypass simple next-record protections
idor-jump-backward	-2	Short jump backward to access older records
idor-jump-medium	+10	Medium jump to test non-sequential horizontal escalation
idor-negative-index	-5	Lower-bound approximation to test negative indexing
idor-far-boundary	+99	Far boundary test to reach old / admin-reserved initial objects

10.3 Test Cases

Target endpoints: GET /documents/<id>, GET /profile/<id>, GET /messages/<id>, GET /api/v1/users/<id> (vulnerable_app).

Test ID	Test Name	Input	Expected Result	Pass Condition
TC-IDOR-01	Adjacent next ID (path)	GET /documents/<id+1>	VULNERABLE	Cross-tenant document returned; signature or structural heuristic fires
TC-IDOR-02	Adjacent previous ID (path)	GET /profile/<id-1>	VULNERABLE	Another user's profile (api_key, role) returned at HTTP 200
TC-IDOR-03	Short jump forward (+3)	GET /messages/<id+3>	VULNERABLE	Message heuristic fires on structural diff from baseline
TC-IDOR-04	Short jump backward (-2)	GET /documents/<id-2>/download	VULNERABLE	Different user's file downloaded without ownership check
TC-IDOR-05	Medium jump (+10)	GET /api/v1/users/<id+10>	VULNERABLE	JSON response contains api_key and role:admin — signature matches
TC-IDOR-06	Negative index (-5)	GET /profile/<id-5>	VULNERABLE	Structural diff >15 bytes vs baseline plus signature match
TC-IDOR-07	Far boundary (+99)	GET /documents/<id+99>	N/A	ID out of range — 404 returned; error-page check rejects it, no finding
TC-IDOR-08	Legitimate own-record access	GET /documents/<own_id>	SAFE	Response matches baseline — no structural difference, no finding raised
TC-IDOR-09	Non-numeric endpoint skipped	GET /dashboard	N/A	No path ID and no id-like parameter — scanner skips the endpoint

11. Integration Testing

Integration tests verify that scanner modules run correctly through the shared async BaseScanner interface, that the Orchestrator collects their results correctly under `asyncio.gather()`, and that findings flow correctly into the FindingRepository, the report builder, and the frontend.

11.1 Shared Finding Structure

Scanners do not return a loose dictionary. Every scanner returns a list of Finding dataclass instances (engine/models/finding.py), each serialized via `.to_dict()` into:

```
{ "id": str, "title": str, "owasp_category": str, "threat_level": str, "cvss_score": str,
  "affected_path": str, "description": str, "business_impact": str,
  "recommendations": [str], "references": [str],
  "proof_of_concept": { "intro_text": str, "steps_to_reproduce": [str],
                        "evidence": { "type": str, "request": str, "response": str } },
  "status": str, // default "Open"
  "timestamp": str // UTC ISO-8601
}
```

11.2 Integration Test Cases

Test ID	Test Scenario	Action	Expected Result	Pass Condition
TC-INT-00	Registry import integrity	Import engine.registry.scanner_registry and instantiate every class in SCANNER_REGISTRY	PASS	All stubs resolved and import typo fixed — SCANNER_REGISTRY loads cleanly and all 8 scanner classes instantiate successfully.
TC-INT-01	Full async scan — vulnerable_app	Run Orchestrator.run_assessment() against the live vulnerable_app	PASS	All 8 scanners (XSS, Authentication, LFI, Path Traversal, File Upload, SQLi, Sensitive File Disclosure, IDOR) report findings; no scanner fails in isolation
TC-INT-02	Full scan — hardened build	Run all runnable scanners against a patched vulnerable_app build	PASS	No findings raised by any scanner
TC-INT-03	Concurrent execution	Confirm all registered scanners run inside the same asyncio.gather() call	PASS	Combined wall-clock time is close to the slowest scanner alone, not the sum of all scanners
TC-INT-04	Exception isolation	Force one scanner to raise inside run_scan() (e.g. the current SQLi scanner)	PASS	return_exceptions=True keeps every other scanner's findings intact
TC-INT-05	Finding structure consistency	Inspect .to_dict() output of every runnable scanner	PASS	All scanners return the same Finding schema (Section 9.1)

Test ID	Test Scenario	Action	Expected Result	Pass Condition
TC-INT-06	Live WebSocket telemetry	Trigger a scan from the frontend	PASS	Log messages from all scanners stream live without blocking the scan
TC-INT-07	Report generation	Feed combined findings to ReportBuilder.build_json_report()	PASS	Valid JSON report produced with every runnable scanner's findings

12. Acceptance Criteria

The following conditions must be met before the final demonstration:

- The two import-breaking blockers in Section 2.4 (orchestrator typo, empty Sensitive File Disclosure stub) are fixed so the engine can start at all.
- The XSS Scanner detects both reflected XSS (on /search) and stored XSS (on /documents/<id>/comment) against vulnerable_app.
- The Authentication Scanner detects missing rate limiting on /login, weak password acceptance on /register, and missing session cookie flags, against vulnerable_app.
- The LFI Scanner detects arbitrary file reads via /preview?template=... against vulnerable_app.
- The Path Traversal Scanner detects arbitrary file reads via /download?file=... against vulnerable_app, including escape to /etc/passwd.
- The File Upload Scanner detects unrestricted upload of executable content via /documents/upload against vulnerable_app.
- All five runnable scanners above return SAFE / no findings when run against a hardened build of vulnerable_app.
- All registered scanners are launched concurrently by the Orchestrator via asyncio.gather() — confirmed by timing, not just by code inspection.
- All findings conform to the shared Finding schema (Section 9.1) and serialize correctly via .to_dict().
- Live scan telemetry streams over WebSockets without stalling or delaying the async scan loop.
- The reporter module generates a valid JSON report from the combined scanner output.
- End-to-end demo (crawl → concurrent scan → correlation → report) runs without errors on the live presentation.
- SQL Injection, Sensitive File Disclosure, and IDOR/Authorization scanners are now fully implemented and validated — all 8 scanners run and report findings during the demo.

13. Sample Scanner Output

The following is a representative Finding produced by the XSS Scanner when run against vulnerable_app's /search endpoint (Reflected XSS), serialized via Finding.to_dict():

```
{
  "id": "VULN-7F3A9C",
  "title": "Reflected Cross-Site Scripting (XSS)",
  "owasp_category": "A03:2021 - Injection",
  "threat_level": "High",
  "cvss_score": "7.2",
  "affected_path": "http://localhost:5004/search [parameter=q]",
  "description": "The application reflects user input from 'q' without proper HTML context encoding.",
  "business_impact": "An attacker can execute arbitrary JavaScript in the victim's browser.",
  "recommendations": ["Encode all user-supplied data before rendering it.", "Implement CSP header."],
  "references": ["https://owasp.org/www-community/attacks/xss/"],
  "proof_of_concept": {
    "intro_text": "Sent structural payload '<script>alert(1)</script>' via parameter 'q'.",
    "steps_to_reproduce": ["1. Target: http://localhost:5004/search", "2. Inject payload into 'q'"],
    "evidence": { "type": "http_snippet",
      "request": "GET /search?q=&lt;script&gt;alert(1)&lt;/script&gt;",
      "response": "Context: Raw Unescaped Reflection Confirmed" }
  },
  "status": "Open",
  "timestamp": "2026-06-30T21:40:00Z"
}
```

Result: the scanner correctly identified the Reflected XSS vulnerability on vulnerable_app and produced a fully-structured Finding ready for the reporter and frontend timeline.

A second representative Finding, from the Path Traversal Scanner against /download?file=../../../../../etc/passwd:

```
{
  "id": "VULN-9B1D42",
  "title": "Path Traversal (Arbitrary File Read)",
  "owasp_category": "A01:2021 - Broken Access Control",
  "threat_level": "High",
  "cvss_score": "7.5",
  "affected_path": "GET /download [Param: file]",
  "description": "Path Traversal detected.",
  "business_impact": "Attackers can read arbitrary files on the server.",
  "recommendations": ["Resolve requested paths and verify containment."],
  "status": "Open"
}
```

14. Tools & Environment

Tool / Environment	Purpose
vulnerable_app (Flask, port 5004)	Main vulnerable target for all scanner testing — replaces DVWA
httpx.AsyncClient	Asynchronous HTTP layer for all scanner requests (replaces requests)
asyncio (Semaphore, gather)	Concurrency control and parallel scanner execution
Playwright (PlaywrightSpider)	Headless-browser crawler for attack-surface discovery
Django + Channels	Backend API and WebSocket layer for live scan telemetry
Python 3.x	Scanner execution environment
pytest (optional)	Automated unit test runner
GitHub	Version control & code review
VSCode	Development environment

15. Appendix — vulnerable_app Bugs With NO Scanner Coverage Yet

Sections 3–8 test every vulnerability class that an actual scanner module can currently detect. But vulnerable_app itself (see the "# VULN:" comments throughout vulnerable_app/app.py) was seeded with several additional, real, exploitable flaws that no scanner module attempts to find at all — not even as an empty stub. They are listed here for transparency and as a backlog for future scanner modules; they are out of scope for this revision's automated pass/fail criteria and must be checked manually if included in a demo.

15.1 Broken Access Control — Missing Role Checks (Admin Panel)

GET /admin/users/export and GET+POST /admin/settings are decorated with @login_required instead of @admin_required, unlike GET /admin which correctly requires admin role. Any authenticated member can export the full user list (including api_key) or change workspace-wide settings. No scanner module currently probes for role-escalation via authenticated-but-unprivileged sessions.

15.2 IDOR (Insecure Direct Object Reference) — Multiple Endpoints

None of the following are covered — the IDOR/Authorization scanner is an empty stub (Section 1.2 / 2.4):

- GET /documents/<id> and GET /documents/<id>/download — no ownership check on private documents.
- GET/POST /documents/<id>/edit — any authenticated user can edit any document's metadata.
- GET/POST /documents/<id>/share and POST /documents/<id>/share/<share_id>/revoke — any authenticated user can share or revoke access to documents they do not own.
- GET /profile/<user_id> — any authenticated user can view any other user's full profile, including their API key.
- GET /messages/<msg_id> — no check that the message belongs to the current user.
- GET /api/v1/users/<user_id> — API equivalent of the profile IDOR; returns any user's data including api_key.

15.3 Username Enumeration (Login)

POST /login returns a different flash message depending on whether the username exists ("Incorrect password..." vs. "No account found for username...") — an attacker can enumerate valid usernames before attempting credential stuffing. No scanner module currently diffs login error messages across a known-good vs. unknown username.

15.4 Verbose Error / Stack-Trace Disclosure

Two separate spots leak internal detail to the client:

- GET /people?department=... — on a SQL error, the response includes the raw exception message, the exception type, and the full interpolated SQL query text (compounds the SQL Injection issue in Section 8).
- The global 500 error handler returns a full Python stack trace, internal file paths, and Flask config details instead of a generic error page.

A "verbose_error" family is already reserved in PayloadManager.FAMILY_MAP (mapped to the "disclosure" folder) and a compiled leftover (verbose_error_scanner.cpython-312.pyc) exists in __pycache__, suggesting a verbose-error scanner existed at some point — but there is no corresponding .py source file in the current codebase and it is not registered in SCANNER_REGISTRY.

15.5 Sensitive Data Exposure via API

- GET /api/v1/user/me returns the caller's own api_key in a plain JSON response — acceptable for the owner, but combined with the IDOR in 13.2 it lets any user pull any other user's key via /api/v1/users/<id>.
- GET /backup/config (Section 2.4 mentions this is the intended target of the still-unimplemented Sensitive File Disclosure scanner) is a completely unauthenticated route — no @login_required at all — serving config.bak to anyone who requests it.

15.6 Summary Table — Coverage Gap

vulnerable_app Flaw	Endpoint(s)	Scanner Status
Broken access control (admin, login-only check)	/admin/users/export, /admin/settings	No scanner exists
IDOR — documents / profile / messages / API	/documents/<id>*, /profile/<id>, /messages/<id>, /api/v1/users/<id>	Empty stub (idor_scanner.py, both copies)
Username enumeration	/login	No scanner exists
Verbose error / stack trace disclosure	/people (SQL error), global 500 handler	No scanner exists (stale .pyc only)
Sensitive data exposure via API	/api/v1/user/me, /api/v1/users/<id>	No scanner exists
Sensitive file disclosure	/backup/config	Empty stub (sensitive_file_scanner.py)